

IF25-22017 PENGEMBANGAN APLIKASI MOBILE

Modul Belajar 2

Model Data, Coroutines, dan Flow

Memodelkan laporan dengan aman, menunggu server tanpa layar macet,
dan mengikuti status yang terus berubah

Muhammad Habib Algifari, S.Kom., M.T.I.

Pendamping Pertemuan 2. Tahun 2026

Komponen	Keterangan
Pertemuan	2
Capaian	CPMK0501 menerapkan dan CPMK0502 menjelaskan konsep pemrograman untuk pengembangan perangkat lunak
Waktu belajar	sekitar 5 jam belajar mandiri, ditambah 2,5 jam di kelas

MODUL 2

Peta belajar

Ikuti urutan ini dari atas ke bawah. Setiap bagian memakai apa yang sudah dibahas di bagian sebelumnya.

Urutan	Bagian
1	Tentang modul ini
2	Masalah pembuka: status laporan berubah, aplikasi harus ikut tahu
3	Langkah 1: data class untuk satu laporan
4	Langkah 2: sealed class untuk status yang terbatas
5	Langkah 3: Null safety, foto boleh kosong
6	Langkah 4: Extension function dan lambda
7	Langkah 5: Kenapa kita butuh coroutines
8	Langkah 6: launch dan async
9	Langkah 7: Dispatcher dan structured concurrency
10	Langkah 8: Flow, aliran nilai dari waktu ke waktu
11	Langkah 9: Operator untuk mengolah aliran
12	Langkah 10: Satu kontrak untuk semua sumber data, dan kalau server gagal
13	Latihan mandiri
14	Bawa ke proyekmu
15	Rangkuman
16	Cek pemahaman
17	Bacaan lanjut dan persiapan minggu depan

PEMBUKA

Tentang modul ini

Modul ini menemani Pertemuan 2. Kita melanjutkan Laporkampus dari Modul 1, jadi pastikan proyek KMP kalian sudah bisa dijalankan. Semua kode di modul ini ditulis di **commonMain**, karena model data dan cara menunggu server berlaku sama untuk semua platform.

Modul ini terdiri dari tiga bagian besar. Langkah 1 sampai 4 membahas cara memodelkan data laporan. Langkah 5 sampai 7 membahas coroutines, yaitu cara menunggu tanpa membuat layar

macet. Langkah 8 sampai 10 membahas Flow, yaitu cara mengikuti data yang terus berubah. Bacalah berurutan, karena setiap bagian memakai hasil bagian sebelumnya.

Setelah menyelesaikan modul ini, kalian bisa

No	Kemampuan	Dibahas di
1	Memodelkan data dengan data class, sealed class, tipe nullable, dan extension function	Langkah 1 sampai 4
2	Menulis suspend function, menjalankan dua proses bersamaan, dan memilih dispatcher	Langkah 5 sampai 7
3	Membuat Flow, mengolahnya dengan operator, dan menangani error	Langkah 8 sampai 10

Capaian ini mendukung **CPMK0501** (menerapkan) dan **CPMK0502** (menjelaskan). Observasi minggu ini punya dua kriteria: kodenya berjalan, dan kalian bisa menjelaskan kenapa hasilnya begitu.

Yang perlu disiapkan

Proyek LaporKampus dari Modul 1 dan Kotlin Playground di play.kotlinlang.org. Playground sudah mendukung `kotlinx.coroutines`, jadi semua contoh di Langkah 5 sampai 10 bisa dicoba di browser tanpa membuka Android Studio. Saat mencoba di Playground, tempelkan juga data class, sealed class, dan fungsi dari langkah sebelumnya, karena setiap contoh memakai yang sudah dibuat sebelumnya.

PENTING Mengingat Modul 1

Kita akan sering memakai `val`, fungsi satu baris, string template, dan `when`. Kalau ada yang terasa asing, buka lagi Langkah 1 sampai 3 di Modul 1 sebelum lanjut.

PEMBUKA

Masalah pembuka: status laporan berubah, aplikasi harus ikut tahu

Minggu lalu kalian menyebutkan data yang wajib ada di satu laporan. Hasilnya kira-kira seperti ini:

Data	Contoh isinya	Catatan
Nomor laporan	7	Selalu ada
Judul	Proyektor mati	Selalu ada
Lokasi	GKU 1 R201	Selalu ada
Foto	boleh kosong	Pelapor tidak selalu sempat memotret
Status	Baru, Diproses, atau Selesai	Pilihannya terbatas

Sekarang bayangkan perjalanan satu laporan. Laporan proyektor masuk pukul 08.10, diterima petugas pukul 08.40, dan selesai pukul 10.15. Ada tiga masalah kecil yang harus diselesaikan aplikasi:

1. **Memodelkan laporan dengan benar**, termasuk kolom yang boleh kosong dan status yang pilihannya terbatas.
2. **Menunggu jawaban server tanpa membuat layar macet.**
3. **Mengetahui setiap kali status berubah**, tanpa pengguna harus menekan tombol muat ulang.

CONTOH Mirip lacak paket

Kalian pasti pernah melacak paket belanja daring. Statusnya bergerak sendiri dari Dikemas, Dikirim, sampai Diterima, dan aplikasi memberi notifikasi setiap kali berubah. Laporkampus butuh hal yang persis sama untuk laporan kerusakan.

LANGKAH 1

data class untuk satu laporan

Untuk menyimpan satu laporan, kita butuh wadah yang mengumpulkan beberapa nilai sekaligus: nomor, judul, lokasi, foto, dan status. Di Kotlin, wadah seperti ini paling mudah ditulis sebagai **data class**.

```
commonMain/model/Laporan.kt
```

```
data class Laporan(
    val id: Int,
    val judul: String,
    val lokasi: String,
    val fotoUrl: String?,
    val status: StatusLaporan = StatusLaporan.Baru
)
```

Status di atas memakai tipe `StatusLaporan` yang baru kita buat di Langkah 2. Untuk sementara, bacalah dulu bentuknya.

ANALOGI Formulir yang sudah dicetak

data class seperti formulir laporan yang kolom-kolomnya sudah dicetak: nomor, judul, lokasi, foto, status. Setiap laporan adalah satu lembar formulir yang sudah diisi.

Apa yang kita dapat dari kata data

Satu kata `data` memberi kita tiga kemampuan tanpa menulisnya sendiri:

Kemampuan	Gunanya	Contoh
<code>toString()</code>	Mencetak isi objek dengan rapi	<code>Laporan(id=7, judul=Proyektor mati, ...)</code>
<code>equals()</code>	Membandingkan isi, bukan alamat objek	Dua laporan dengan isi sama dianggap sama
<code>copy()</code>	Menyalin objek sambil mengganti sebagian nilai	Laporan yang sama dengan status baru

Kenapa semua properti memakai val

Laporan tidak diubah di tempat. Kalau statusnya berubah, kita membuat salinan baru dengan `copy`.

Main.kt

```
val a = Laporan(7, "Proyektor mati", "GKU 1 R201", null)
val b = a.copy(judul = "Proyektor mati total")
println(a.judul)    // Proyektor mati
println(b.judul)    // Proyektor mati total
```

ANALOGI Fotokopi dengan koreksi

`copy` seperti memfotokopi formulir lalu mengoreksi satu kolom di fotokopinya. Formulir aslinya tetap utuh. Karena data lama tidak pernah berubah diam-diam, kita selalu bisa melacak dari mana sebuah nilai berasal. Kebiasaan ini sangat membantu di Pertemuan 4 saat membahas state.

LANGKAH 2

sealed class untuk status yang terbatas

Status laporan hanya punya tiga kemungkinan. Kalau kita menyimpannya sebagai String, masalahnya cepat muncul: "baru", "Baru", dan "BARU" dianggap berbeda, dan salah ketik seperti "Diprosess" tidak ketahuan sampai aplikasi dipakai orang.

Solusinya adalah **sealed class**: kelas yang daftar turunannya tertutup dan diketahui compiler.

commonMain/model/StatusLaporan.kt

```
sealed class StatusLaporan {
    data object Baru : StatusLaporan()
    data class Diproses(val petugas: String) : StatusLaporan()
    data class Selesai(val catatan: String) : StatusLaporan()
}
```

Perhatikan bahwa sebagian status membawa data sendiri. Status Diproses menyimpan nama petugas, status Selesai menyimpan catatan penyelesaian. Status Baru tidak membawa data apa pun, jadi cukup ditulis sebagai `data object`, yaitu satu objek tunggal.

ANALOGI Pilihan ganda, bukan isian bebas

String seperti soal isian bebas: orang bisa menulis apa saja, termasuk yang salah. sealed class seperti soal pilihan ganda: jawabannya hanya A, B, atau C, dan pilihan D tidak mungkin dipilih.

when yang memeriksa semua kemungkinan

Karena daftarnya tertutup, `when` bisa memastikan semua status sudah ditangani. Kita tidak perlu cabang `else`.

Main.kt

```
fun warna(s: StatusLaporan): String = when (s) {
    StatusLaporan.Baru → "abu-abu"
    is StatusLaporan.Diproses → "kuning"
    is StatusLaporan.Selesai → "hijau"
}
```

Kata `is` memeriksa jenis objek. Untuk Baru kita tidak memakai `is` karena ia satu objek tunggal, jadi cukup dibandingkan langsung.

TEBAK DULU

Suatu hari kita menambah status keempat, data `class Ditolak(val alasan: String)`. Apa yang terjadi pada fungsi `warna` di atas?

JAWABAN

Kode gagal dikompilasi, karena `when` tidak lagi menangani semua kemungkinan. Compiler langsung menunjuk setiap `when` yang belum punya cabang `Ditolak`. Inilah kekuatan sealed class: lupa menangani satu status menjadi error saat compile, bukan bug saat aplikasi dipakai.

LANGKAH 3

Null safety, foto boleh kosong

Di Modul 1 kita sudah berkenalan dengan tanda tanya pada tipe. Sekarang kita pakai dengan sungguh-sungguh. Di data class `Laporan`, hanya `fotoUrl` yang bertipe `String?`, karena hanya foto yang boleh kosong.

Tanda tanya itu adalah pengakuan jujur bahwa nilainya mungkin kosong. Sebagai gantinya, Kotlin memaksa kita menangani kemungkinan kosong itu sebelum memakai nilainya.

Main.kt

```
val foto: String? = laporan.fotoUrl
val adaFoto = foto?.isNotBlank()           // null kalau foto kosong
val teks = foto ?: "Belum ada foto"        // nilai cadangan
val panjang = foto!!.length                // berbahaya, lihat penjelasan
```

Penulisan	Artinya	Kapan dipakai
<code>String?</code>	Nilainya boleh null	Kolom yang memang opsional
<code>?.</code>	Jalankan hanya kalau tidak null	Mengambil sesuatu dari nilai yang mungkin kosong
<code>?:</code>	Pakai nilai cadangan kalau null	Menampilkan teks pengganti
<code>!!</code>	Saya yakin tidak null, kalau salah aplikasi crash	Hampir tidak pernah di proyek kita

CONTOH Contoh sehari-hari

Bayangkan kalian menitip pesan ke teman: tolong ambilkan paket di pos satpam, kalau ada. ?. artinya teman kalian mengecek dulu, dan kalau paketnya tidak ada ia pulang dengan tangan kosong tanpa masalah. ?: artinya kalau paketnya tidak ada, belikan saja air minum sebagai gantinya. !! artinya kalian bersikeras paketnya pasti ada, dan kalau ternyata tidak ada, teman kalian marah besar. Itulah crash.

PENTING Hindari tanda seru ganda

!! adalah sumber crash yang paling sering di aplikasi Kotlin. Baris terakhir di contoh atas sengaja ditulis untuk dibahas, bukan untuk ditiru. Di proyek kita, pakai ?. dan ?!.

LANGKAH 4

Extension function dan lambda

Menambah kemampuan tanpa mengubah class

Kita ingin setiap status punya label yang enak dibaca, dan setiap laporan punya ringkasan satu baris. Kita bisa menempelkan fungsi baru ke tipe yang sudah ada, tanpa mengubah class aslinya. Fungsi seperti ini disebut **extension function**.

```
commonMain/model/LaporanExt.kt
```

```
fun StatusLaporan.label(): String = when (this) {
    StatusLaporan.Baru → "Menunggu petugas"
    is StatusLaporan.Diproses → "Ditangani $petugas"
    is StatusLaporan.Selesai → "Selesai: $catatan"
}

fun Laporan.ringkas(): String = "#$id $judul, $lokasi"
```

Di dalam extension function, `this` adalah objek yang memanggilnya. Karena itu kita bisa langsung menulis `$id` dan `$judul`. Perhatikan juga: setelah `is StatusLaporan.Diproses`, Kotlin tahu objeknya pasti `Diproses`, sehingga `$petugas` bisa dipakai langsung. Fitur ini disebut **smart cast**.

ANALOGI Casing HP

Extension function seperti memasang casing dengan tempat kartu di HP kalian. HP-nya tidak dibongkar, tetapi sekarang punya kemampuan tambahan. Pemanggilannya pun terasa bawaan: `laporan.ringkas()`.

Lambda: fungsi tanpa nama

Main.kt

```
val belumDitangani = semuaLaporan.filter { it.status is StatusLaporan.Baru }
```

Blok di dalam kurung kurawal adalah **lambda**, yaitu fungsi tanpa nama yang dikirim sebagai argumen. `filter` memanggil lambda itu untuk setiap laporan, dan hanya meloloskan laporan yang hasilnya true. Kata `it` adalah nama bawaan untuk satu laporan yang sedang diperiksa.

Pola lambda ini akan kita pakai terus, di Flow minggu ini dan di Compose minggu depan. Detailnya ada di Kotlin Dasar P5 Functional Programming.

TEBAK DULU

Tulis empat baris keluaran program ini sebelum menjalankannya. Apakah `copy` mengubah laporan a?

Main.kt

```
val a = Laporan(7, "Proyektor mati", "GKU 1 R201", null)
val b = a.copy(status = StatusLaporan.Diproses("Pak Joko"))
println(a == b)
println(a.status.label())
println(b.status.label())
println(a.fotoUrl?.length ?: 0)
```

JAWABAN

Keluaran

```
false
Menunggu petugas
Ditangani Pak Joko
0
```

Laporan a tidak berubah karena `copy` membuat objek baru. `a == b` bernilai false karena statusnya berbeda, dan data class membandingkan isi. Baris terakhir bernilai 0: `fotoUrl` kosong, jadi `?.length` menghasilkan null, lalu `?:` menggantinya dengan 0.

LANGKAH 5

Kenapa kita butuh coroutines

Mengambil data laporan dari server bisa memakan waktu satu sampai dua detik. Kalau proses itu berjalan di thread yang sama dengan tampilan, layar membeku, tombol tidak merespons, dan Android akhirnya menampilkan dialog aplikasi tidak merespons.

Coroutine membuat kita menulis kode yang menunggu secara berurutan, seperti biasa, tanpa menahan thread yang sedang dipakai.

ANALOGI Pelayan rumah makan

Pelayan yang kurang cekatan menunggu di satu meja sampai masakannya matang, baru pindah ke meja lain. Pelayan yang cekatan mencatat pesanan, melayani meja lain selagi menunggu, lalu kembali saat masakan siap. Thread adalah pelayannya, coroutine adalah cara kerjanya yang cekatan. Ribuan coroutine bisa berjalan di beberapa thread saja.

Menambahkan library coroutines

Coroutines tersedia lewat library `kotlinx.coroutines`. Daftarkan di `gradle/libs.versions.toml`, lalu pakai di build file modul.

composeApp/build.gradle.kts

```
kotlin {
    sourceSets {
        commonMain.dependencies {
            implementation(libs.kotlinx.coroutines.core)
        }
    }
}
```

Setelah menyimpan, klik Sync Now di Android Studio. Versi library selalu dicatat di `libs.versions.toml`, bukan langsung di `build.gradle.kts`, supaya semua versi ada di satu tempat.

Ini peta istilah yang akan kita pakai di Langkah 5 sampai 7:

Istilah	Fungsinya
<code>suspend</code>	Menandai fungsi yang boleh berhenti sejenak untuk menunggu
<code>launch</code>	Memulai coroutine yang tidak mengembalikan hasil
<code>async</code>	Memulai coroutine yang mengembalikan hasil
Dispatchers	Menentukan thread mana yang bekerja
<code>CoroutineScope</code>	Wadah yang mengatur umur coroutine

suspend function, fungsi yang bisa menunggu

Server kita masih pura-pura. Kita memakai `delay` untuk meniru waktu tunggu jaringan.

```
commonMain/data/LaporanSumber.kt

suspend fun ambilLaporan(id: Int): Laporan {
    delay(1000) // pura-pura menunggu server
    return Laporan(id, "Proyektor mati", "GKU 1 R201", null)
}

suspend fun ambilRiwayat(id: Int): List<String> {
    delay(800)
    return listOf("Dikirim 08.10", "Diterima 08.40")
}
```

`delay` menunda coroutine, bukan thread. Selama menunggu, thread bebas mengerjakan hal lain. Di Pertemuan 6, isi fungsi ini diganti dengan panggilan ke server sungguhan, dan kode yang memanggilnya tidak perlu berubah sama sekali.

PENTING Satu aturan suspend

suspend function hanya bisa dipanggil dari coroutine atau dari suspend function lain. Kalau kalian memanggilnya dari fungsi biasa, muncul error **Suspend function can only be called from a coroutine or another suspend function.**

LANGKAH 6

launch dan async

Ada dua cara memulai coroutine. `launch` dipakai untuk menyuruh: kerjakan ini, aku tidak butuh hasilnya. `async` dipakai untuk meminta hasil: kerjakan ini, nanti aku ambil hasilnya dengan `await`.

```
Main.kt

import kotlinx.coroutines.*

fun main() = runBlocking {
    launch { println("Notifikasi terkirim") }
    val laporan = async { ambilLaporan(7) }
    val riwayat = async { ambilRiwayat(7) }
    println(laporan.await().judul)
    println(riwayat.await())
}
```

`launch`

`async`

Hasil

Tidak ada

Deferred, diambil dengan `await()`

	launch	async
Dipakai untuk	Tugas tanpa jawaban, misalnya mengirim notifikasi	Mengambil beberapa data bersamaan

`runBlocking` membuat jembatan dari fungsi main biasa ke dunia coroutine. Ia hanya dipakai di fungsi main dan di pengujian, tidak di aplikasi.

TEBAK DULU

ambilLaporan menunggu 1000 ms dan ambilRiwayat menunggu 800 ms. Berapa lama program di atas berjalan: sekitar 1800 ms, 1000 ms, atau 800 ms? Lalu baris mana yang tercetak paling dulu?

JAWABAN

Sekitar **1000 ms**. Kedua `async` berjalan bersamaan, jadi kita hanya menunggu yang paling lama. Baris pertama yang tercetak adalah Notifikasi terkirim, karena saat main menunggu `await()`, coroutine dari `launch` mendapat giliran bekerja.

ANALOGI Pesan ojek sambil pesan makanan

Kalian lapar dan harus berangkat kuliah. Cara lambat: pesan makanan, tunggu sampai datang, baru pesan ojek. Cara cepat: pesan keduanya sekaligus, lalu tunggu yang datang paling lama. `async` adalah cara cepat itu.

Buktikan dengan mengukur waktu

Main.kt

```
import kotlin.system.measureTimeMillis
import kotlinx.coroutines.*

fun main() = runBlocking {
    val bersamaan = measureTimeMillis {
        val a = async { ambilLaporan(7) }
        val b = async { ambilRiwayat(7) }
        a.await(); b.await()
    }
    val berurutan = measureTimeMillis {
        ambilLaporan(7)
        ambilRiwayat(7)
    }
    println("Bersamaan: $bersamaan ms")
    println("Berurutan: $berurutan ms")
}
```

Keluaran, angkanya bisa sedikit berbeda

Bersamaan: 1012 ms
Berurutan: 1806 ms

PENTING Coba sendiri

Ganti `delay(800)` di `ambilRiwayat` menjadi `delay(1500)`, lalu jalankan lagi. Hasilnya kira-kira 1500 ms untuk bersamaan dan 2500 ms untuk berurutan. `measureTimeMillis` hanya tersedia di JVM, jadi pakai di Playground atau proyek latihan, bukan di `commonMain`. Latihan 1 di repo meminta hal yang sama.

LANGKAH 7

Dispatcher dan structured concurrency

Dispatcher menentukan siapa yang bekerja

Dispatcher	Dipakai untuk	Contoh di LaporanKampus
Main	Memperbarui tampilan	Menampilkan daftar laporan
IO	Jaringan, database, berkas	Mengambil laporan dari server
Default	Hitungan berat	Mengurutkan ribuan laporan

`commonMain/data/LaporanSumber.kt`

```
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.IO
import kotlinx.coroutines.withContext

suspend fun muatDetail(id: Int): Laporan =
    withContext(Dispatchers.IO) {
        ambilLaporan(id)
    }
```

`withContext` memindahkan pekerjaan ke dispatcher lain, lalu kembali membawa hasilnya. Di `commonMain`, `Dispatchers.IO` butuh baris `import kotlinx.coroutines.IO`.

ANALOGI Loket di kantor layanan

Dispatcher seperti pembagian loket. Loket depan (Main) hanya melayani tamu dan tidak boleh ditinggal lama. Urusan berkas yang lama diserahkan ke bagian arsip (IO). Hitungan rumit dikerjakan bagian keuangan (Default). Petugas loket depan tinggal menerima hasilnya.

Structured concurrency, setiap coroutine punya induk

Coroutine di Kotlin tidak berjalan lepas begitu saja. Ada empat aturan yang selalu berlaku:

Aturan	Artinya
Punya induk	Setiap coroutine berjalan di dalam satu scope
Induk menunggu	Scope belum selesai sebelum semua anaknya selesai
Batal bersama	Kalau induk dibatalkan, semua anaknya ikut berhenti
Gagal diteruskan	Kalau satu anak gagal, kegagalannya diteruskan ke induk

CONTOH Contoh sehari-hari

Bayangkan rombongan kunjungan industri dengan satu ketua rombongan. Ketua belum boleh pulang sebelum semua anggota kembali ke bus. Kalau kunjungan dibatalkan, semua anggota ikut pulang. Kalau ada anggota yang sakit, ketua langsung diberi tahu. Tidak ada anggota yang tertinggal di lokasi tanpa ada yang tahu.

Di Laporkampus, saat pengguna menutup layar detail laporan, semua pengambilan data untuk layar itu ikut dibatalkan. Tidak ada proses yang terus berjalan diam-diam dan memboroskan baterai. Karena itu, hindari `GlobalScope`: coroutine di sana tidak punya induk yang bisa membatalkannya. Di Pertemuan 4 kita akan memakai `viewModelScope` yang mengikuti umur layar.

LANGKAH 8

Flow, aliran nilai dari waktu ke waktu

`suspend function` mengembalikan **satu** nilai, sekali, lalu selesai. Itu cocok untuk membuka detail satu laporan. Tetapi status laporan berubah beberapa kali. Untuk itu kita butuh **Flow**: aliran yang memancarkan beberapa nilai berurutan dari waktu ke waktu.

	<code>suspend function</code>	Flow
Jumlah nilai	Satu	Beberapa, berurutan
Contoh tanda tangan	<code>suspend fun ambilLaporan(id: Int): Laporan</code>	<code>fun pantauStatus(id: Int): Flow<StatusLaporan></code>
Cocok untuk	Detail satu laporan	Status yang berubah dari Baru sampai Selesai

Membuat Flow

```
commonMain/data/LaporanSumber.kt
```

```
import kotlinx.coroutines.delay
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.flow

fun pantauStatus(id: Int): Flow<StatusLaporan> = flow {
    emit(StatusLaporan.Baru)
    delay(1500)
    emit(StatusLaporan.Diproses("Pak Joko"))
    delay(2000)
    emit(StatusLaporan.Selesai("Lampu proyektor diganti"))
}
```

Di dalam blok `flow { }`, kita memanggil `emit` setiap kali ada nilai baru. Ada juga cara yang lebih singkat untuk nilai yang sudah diketahui: `flowOf(a, b, c)` atau `daftar.asFlow()`.

Perhatikan bahwa `pantauStatus` tidak memakai kata `suspend`. Membuat Flow tidak menunggu apa-apa. Yang menunggu adalah pihak yang mengumpulkan nilainya.

collect membuka keran

Main.kt

```
fun main() = runBlocking {
    val aliran = pantauStatus(7)
    println("Sebelum collect")
    aliran.collect { status →
        println(status.label())
    }
    println("Sesudah collect")
}
```

Keluaran

```
Sebelum collect
Menunggu petugas
Ditangani Pak Joko
Selesai: Lampu proyektor diganti
Sesudah collect
```

Ada dua hal penting di sini. Pertama, baris `val aliran = pantauStatus(7)` belum menjalankan apa pun. Flow bersifat **cold**: ia diam sampai ada yang memanggil `collect`. Kedua, baris `Sesudah collect` baru tercetak setelah Flow selesai, karena `collect` ikut menunggu sampai nilai terakhir.

ANALOGI Video sesuai permintaan

Flow yang cold seperti video di platform streaming, bukan siaran televisi. Video baru diputar saat kalian menekan tombol play, dan setiap penonton menonton dari awal. Begitu pula Flow: setiap `collect` memulai aliran dari awal.

TEBAK DULU

Apa yang terjadi kalau kita memanggil `aliran.collect { ... }` dua kali berturut-turut?

JAWABAN

Ketiga status dicetak dua kali, dan total waktunya menjadi sekitar dua kali lipat. Setiap `collect` menjalankan isi blok flow dari awal, termasuk semua delay di dalamnya.

LANGKAH 9

Operator untuk mengolah aliran

Nilai di Flow bisa diolah di tengah jalan sebelum sampai ke `collect`. Alat pengolahnya disebut **operator**.

Operator	Gunanya
<code>map</code>	Mengubah setiap nilai
<code>filter</code>	Meloloskan nilai yang memenuhi syarat
<code>take</code>	Mengambil beberapa nilai pertama saja
<code>onEach</code>	Efek samping tanpa mengubah nilai, misalnya mencatat log
<code>catch</code>	Menangani error yang terjadi di aliran

Main.kt

```
fun main() = runBlocking {
    pantauStatus(7)
        .filter { it !is StatusLaporan.Baru }
        .map { it.label() }
        .onEach { println("Log: $it") }
        .collect { println("Notifikasi: $it") }
}
```

Nilai mengalir dari atas ke bawah. `filter` membuang status Baru karena kita hanya peduli perubahan. `map` mengubah status menjadi kalimat. `onEach` mencatat log. `collect` menampilkan notifikasi.

ANALOGI Saluran air

Bayangkan saluran air dari sumber ke bak penampung, dengan saringan dan keran di tengahnya. `pantauStatus` adalah sumbernya, `filter` adalah saringannya, `map` adalah alat yang mengubah warna air, dan `collect` adalah keran di ujung. Selama keran belum dibuka, tidak ada air yang mengalir sama sekali.

TEBAK DULU

Berapa kali baris Notifikasi tercetak? Tulis seluruh keluarannya.

JAWABAN

Dua kali, karena status Baru dibuang oleh filter.

Keluaran

```
Log: Ditangani Pak Joko
Notifikasi: Ditangani Pak Joko
Log: Selesai: Lampu proyektor diganti
Notifikasi: Selesai: Lampu proyektor diganti
```

Perhatikan bahwa Log dan Notifikasi bergantian. Setiap nilai melewati seluruh rantai operator sampai ke `collect`, baru nilai berikutnya menyusul.

LANGKAH 10

Satu kontrak untuk semua sumber data, dan kalau server gagal

interface dengan tipe generik

Sejauh ini fungsi `ambilLaporan` dan `pantauStatus` berdiri sendiri. Kita satukan dalam satu kontrak, supaya nanti sumber palsu bisa diganti dengan server sungguhan tanpa mengubah kode pemanggil.

```
commonMain/data/Sumber.kt
```

```

interface Sumber<T> {
    suspend fun ambil(id: Int): T
    fun pantau(id: Int): Flow<T>
}

fun contohLaporan(id: Int) = Laporan(id, "Proyektor mati", "GKU 1 R201", null)

class SumberLaporanPalsu : Sumber<Laporan> {
    override suspend fun ambil(id: Int): Laporan {
        delay(1000)
        return contohLaporan(id)
    }
    override fun pantau(id: Int): Flow<Laporan> = flow {
        emit(contohLaporan(id))
    }
}

```

interface adalah janji: siapa pun yang mengaku sebagai Sumber wajib punya fungsi ambil dan pantau. Kata **override** menandai fungsi yang menepati janji itu. **<T>** adalah **tipe generik**, yang membuat janji yang sama berlaku untuk Laporan, Ruang, Buku, atau entitas tema kelompok kalian.

CONTOH Contoh sehari-hari

Colokan listrik di dinding adalah interface. Charger HP, setrika, dan kipas angin berbeda isinya, tetapi semuanya menepati bentuk colokan yang sama, jadi bisa dipakai di stopkontak mana pun. Di Pertemuan 6 kita akan menulis SumberLaporanServer. Kode pemanggil tidak tahu bedanya, sama seperti stopkontak yang tidak peduli alat apa yang dicolokkan.

Kalau server gagal: try, catch, dan pembatalan

```

commonMain/data/MuatAman.kt

import kotlinx.coroutines.CancellationException

val sumber: Sumber<Laporan> = SumberLaporanPalsu()

suspend fun muatAman(id: Int): String =
    try {
        require(id > 0) { "id harus positif" }
        sumber.ambil(id).ringkas()
    } catch (e: CancellationException) {
        throw e // pembatalan jangan ditelan
    } catch (e: Exception) {
        "Gagal memuat laporan #$id"
    }

```

Ada tiga hal yang perlu diperhatikan. Pertama, **try** di Kotlin menghasilkan nilai, jadi bisa langsung dipakai sebagai isi fungsi. Kedua, **require** melempar error kalau syaratnya tidak terpenuhi, jadi **muatAman(0)** menghasilkan Gagal memuat laporan #0. Ketiga, **CancellationException** harus dilempar ulang.

PENTING Kenapa CancellationException dilempar ulang

Saat coroutine dibatalkan, Kotlin memberi tahu lewat CancellationException. Kalau exception ini ikut ditangkap oleh `catch (e: Exception)` lalu ditelan, coroutine yang seharusnya berhenti malah terus berjalan, dan aturan batal bersama di Langkah 7 rusak. Karena itu, tangkap CancellationException lebih dulu dan lempar lagi.

Untuk error di dalam Flow, pakai operator `catch`:

Main.kt

```
pantauStatus(7)
    .map { it.label() }
    .catch { emit("Status tidak bisa dimuat") }
    .collect { println(it) }
```

PRAKTIK

Latihan mandiri

Ada tiga latihan di repo mata kuliah. Latihan 1 dan 2 adalah latihan yang diobservasi.

Latihan	Lokasi	Yang dilatih
1	P2 Advanced Kotlin Coroutines Flow Hands-on, handson1-latihan	Mengambil profil dan postingan bersamaan. Targetnya sekitar 1000 ms, bukan 1800 ms
2	P2 Advanced Kotlin Coroutines Flow Hands-on, handson2-latihan	Menyaring suhu di atas 30 derajat, mengubahnya menjadi pesan peringatan, lalu menampilkannya
3	Kotlin Dasar, P2 Object-Oriented Programming Hands-on, handson3-latihan	sealed class NetworkResult dengan Loading, Success, dan Error, ditangani dengan when

CONTOH Petunjuk latihan 1

Polanya sama persis dengan Langkah 6. Mulai kedua pengambilan dengan `async`, baru panggil `await()` setelah keduanya dimulai. Kesalahan yang sering terjadi adalah menulis `async { ... }.await()` dalam satu baris, sehingga pengambilan kedua baru dimulai setelah yang pertama selesai, dan waktunya kembali menjadi 1800 ms.

CONTOH Petunjuk latihan 2

Susun rantainya seperti Langkah 9: `filter` untuk suhu di atas 30, `map` untuk mengubah angka menjadi kalimat peringatan, lalu `collect` untuk menampilkan. Ujilah dengan suhu tepat 30 derajat. Lebih dari 30 berarti nilai 30 tidak ikut lolos.

CONTOH Petunjuk latihan 3

Latihan ini sengaja belum bisa dikompilasi sebelum bagian `TODO` dilengkapi, jadi wajar kalau awalnya merah. Tiru pola `StatusLaporan` di Langkah 2: `Loading` tanpa data sebagai data `object`, `Success` dan `Error` sebagai data `class` yang membawa datanya.

Yang diamati

Kriteria	Sangat baik, 85 sampai 100	Baik, 70 sampai 84	Cukup, 55 sampai 69	Kurang, di bawah 55
Coroutines dan Flow berfungsi, CPMK0501, 2%	Latihan 1 selesai sekitar 1000 ms dan latihan 2 menyaring serta memetakan dengan benar	Keduanya jalan, satu hasil kurang tepat	Hanya satu latihan yang jalan	Belum ada yang jalan
Menjelaskan coroutines dan Flow, CPMK0502, 2%	Menjelaskan kenapa async lebih cepat dan kapan Flow mulai mengalir, dengan kode sendiri	Penjelasan benar, ada istilah yang keliru	Hanya membacakan kode	Tidak bisa menjelaskan

Observasi dikerjakan individu dengan bobot 4% dari nilai akhir. Buktinya: push latihan ke repository latihan pribadi, lalu kirim tautan commit di LMS sebelum kelas selesai. Saat berkeliling, saya akan meminta beberapa dari kalian menjelaskan kenapa latihan 1 hanya butuh sekitar 1000 ms.

Kalau error

Pesan error	Penyebab	Yang dicoba
Suspend function can only be called from a coroutine	Memanggil suspend function dari fungsi biasa	Panggil dari dalam launch, runBlocking, atau scope lain
Flow invariant is violated	Memanggil launch di dalam blok flow	Pakai channelFlow untuk pancaran dari beberapa coroutine
JobCancellationException	Scope induk dibatalkan sebelum anaknya selesai	Periksa umur scope, rapikan dengan try dan finally
Unresolved reference: IO	Import Dispatchers.IO di commonMain belum ada	Tambahkan <code>import kotlinx.coroutines.IO</code>

PRAKTIK

Bawa ke proyekmu

Sekarang terapkan Langkah 1 sampai 4 pada tema kelompok kalian. Kerjakan di repository kelompok, folder commonMain/model.

1. **Entitas utama.** Satu data class untuk entitas utama tema kalian, lengkap dengan tipe datanya.
2. **Entitas pendukung.** Data class kedua di tema kalian, misalnya Ruang atau MataKuliah.
3. **Satu sealed class** untuk status dengan tiga nilai sesuai Bank Proyek, misalnya Diajukan, Disetujui, dan Ditolak.
4. **Tandai yang boleh kosong.** Pakai tipe nullable hanya untuk kolom yang memang boleh kosong.

CONTOH Contoh dari LaporanKampus

Laporan adalah entitas utamanya, StatusLaporan adalah sealed class-nya, dan fotoUrl satu-satunya kolom nullable. Sebelum menandai sebuah kolom nullable, tanyakan: apakah data ini benar-benar boleh tidak ada? Kalau jawabannya tidak, jangan diberi tanda tanya.

Checkpoint minggu 2

No	Yang diselesaikan
1	Dua data class dan satu sealed class status di commonMain/model
2	Implementasi Sumber untuk entitas utama, dengan delay dan Flow perubahan status
3	Fungsi main atau test kecil yang mencetak perubahan status ke konsol

Push ke repository kelompok. README memuat contoh keluaran konsol dan siapa mengerjakan bagian mana. Checkpoint tidak masuk nilai, tetapi menjadi bekal tes lisan di pertemuan 12 sampai 16. Model data dan sumber palsu ini langsung dipakai layar daftar minggu depan.

PENUTUP

Rangkuman

data class mengumpulkan beberapa nilai dalam satu wadah dan memberi kita toString, equals, dan copy. Pakai val, lalu ubah dengan copy.

sealed class membatasi pilihan, sehingga `when` bisa memeriksa semua kemungkinan dan salah ketik tertangkap saat compile. Status tanpa data ditulis sebagai `data object`.

Tipe nullable ditulis dengan tanda tanya dan ditangani dengan `?.` dan `?:`. Hindari `!!`.

Extension function menambah kemampuan ke tipe yang sudah ada. **Lambda** adalah fungsi tanpa nama yang dikirim sebagai argumen, seperti di `filter { }`.

Coroutine membuat kita menunggu tanpa menahan thread. suspend function hanya dipanggil dari coroutine. `launch` untuk tugas tanpa hasil, `async` dan `await` untuk mengambil beberapa data bersamaan. Dispatcher menentukan thread yang bekerja, dan setiap coroutine punya induk yang mengatur umurnya.

Flow memancarkan beberapa nilai dari waktu ke waktu. Flow bersifat cold dan baru mengalir saat `collect` dipanggil. Operator seperti `filter`, `map`, `onEach`, dan `catch` mengolah nilai di tengah jalan.

interface generik menyatukan cara mengambil data, sehingga sumber palsu bisa diganti server tanpa mengubah pemanggilnya. Di dalam coroutine, CancellationException selalu dilempar ulang.

PENUTUP

Cek pemahaman

Jawab tanpa melihat modul dulu, lalu cocokkan dengan kunci di bawahnya.

1. Kenapa status laporan lebih aman ditulis sebagai sealed class daripada String?
2. Laporan a dibuat, lalu `val b = a.copy(judul = "AC bocor")`. Berapa nilai `a.judul` sekarang?
3. Apa hasil `val x: String? = null; println(x?.length ?: -1)`?
4. Kenapa dua async yang menunggu 1000 ms dan 800 ms selesai dalam sekitar 1000 ms?
5. Apa yang terjadi pada sebuah Flow kalau tidak ada yang memanggil `collect`?
6. Di rantai `filter`, `map`, `collect` pada Langkah 9, kenapa status Baru tidak pernah sampai ke `collect`?
7. Kenapa `CancellationException` harus dilempar ulang di dalam `catch`?

JAWABAN Kunci jawaban

1. Pilihannya tertutup, jadi salah ketik langsung ketahuan saat compile, dan `when` bisa memastikan semua status sudah ditangani.
2. Tetap Proyektor mati. `copy` membuat objek baru dan tidak mengubah objek asli.
3. `-1`. `x` kosong, jadi `x?.length` menghasilkan `null`, lalu `?:` menggantinya dengan `-1`.
4. Keduanya berjalan bersamaan, jadi total waktunya mengikuti yang paling lama.
5. Tidak ada yang berjalan sama sekali, karena Flow bersifat cold.
6. Karena filter hanya meloloskan nilai yang bukan Baru. Nilai yang tidak lolos tidak diteruskan ke operator berikutnya.
7. Kalau ditelan, coroutine yang seharusnya batal terus berjalan, sehingga aturan batal bersama pada structured concurrency rusak.

PENUTUP

Bacaan lanjut dan persiapan minggu depan

Sumber	Untuk apa
kotlinlang.org, Data classes	Detail equals, copy, dan aturan data class
kotlinlang.org, Sealed classes and interfaces	Sealed class dan when yang lengkap
kotlinlang.org, Null safety	Semua operator untuk nilai yang boleh kosong
kotlinlang.org, Coroutines basics	launch, async, dan scope
kotlinlang.org, Asynchronous Flow	Flow dan operatornya
Kotlin Dasar P2, P3, P5, dan P8	OOP, generics, lambda, dan exception

CONTOH Masalah Pertemuan 3

Data laporan sudah ada. Sekarang bagaimana menampilkannya sebagai daftar kartu yang rapi, dengan judul, lokasi, dan label status, di layar Android dan desktop sekaligus? Bawa model data dan sumber palsu kelompok kalian. Fungsi `Label()` dari Langkah 4 akan langsung dipakai sebagai teks di kartu laporan.